Transductive Generative Modeling: Learning Robot Policies that can Extend to Unseen Conditions

Yating Lin, Necmiye Ozay[†], Yixin Wang[†]
University of Michigan

Abstract: Conditional generative modeling enables learning multimodal robotic behaviors from demonstrations, while robust generalization to unseen conditions outside the training distribution remains a challenge. In this work, we propose Transductive Generative Modeling (TGM), an algorithm that leverages a transductive reparameterization of conditional generative models for Out-of-Support (OOS) generalization. This reparameterization allows some OOS conditions, which we call *delta-representable*, to be represented by an in-support anchor and an in-support displacement. TGM combines the anchor and displacement through bilinear transduction, which can be interpreted as a learnable Taylor expansion around the anchor. This interpretation motivates extending transductive prediction to conditional generative models and reveals the structural assumptions. We instantiate TGM across multiple robotics tasks, including navigation and manipulation, and evaluate its performance on different unseen conditions. Across all tasks, TGM outperforms existing methods and demonstrates more robust generalization under unseen conditions.

1 Introduction

Imitation learning lets robots acquire navigation and manipulation skills from demonstrations. Conditional generative models, such as denoising diffusion probabilistic models (DDPM) [1] and flow-matching models [2], strengthen the capability to capture diverse behaviors from demonstrations, yet generating successful behavior under distribution shifts remains difficult. This challenge is particularly relevant to robotics scenarios, where variations in state, task objectives, physical parameters, and environmental configurations induce distribution shifts between training and deployment. Addressing this challenge calls for learned models that preserve the capability to model expressive distributions of demonstrations while generalizing to conditions beyond the support of training data.

To address this challenge, several previous studies within and beyond robotics have explored different inductive biases that generalize model predictions outside the training support. These approaches exploit structure such as geometric equivariance [3] and periodicity [4] to relate predictions under observed and unseen inputs. However, distribution shifts may extend beyond those captured by the imposed structure. Transductive learning introduces another form of structure by representing unseen inputs through their relationships to training anchors [5]. Within conditional generative modeling, Rubbi et al. [6] improve generalization to unseen conditions by jointly learning a condition-dependent mixture base distribution and a conditional flow field in computational biology. These approaches highlight the role of structural inductive biases in extrapolation and motivate their extending into imitation learning for generalization to unseen conditions in robotics.

We study robotic imitation under Out-of-Support (OOS) conditions, a type of Out-of-Distribution (OOD) shift in which test conditions lie outside the training support. The robot needs to produce an action or trajectory for a condition absent from the demonstration support while preserving multiple valid behaviors when they exist. A change in goal location or in a physical parameter observed between two training conditions may relate an unseen condition to a demonstrated one. The central question is how a robot can reuse these observed changes to act under an unseen combination of conditions.

[†]Equal advising.

CoRL 2026 Workshop on The Action Gap (TAG), Austin TX, USA.

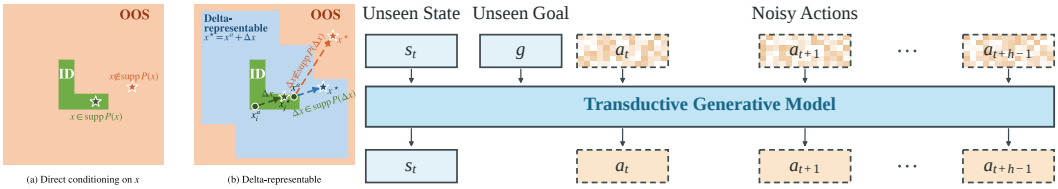


Figure 1: **Left:** In-distribution (ID), Delta-representable, and OOS regions for an L-shaped training support. **Right:** TGM action prediction.

Main idea. We propose Transductive Generative Modeling (TGM), a policy learning method for this robotic extrapolation problem. TGM learns how generated behavior changes between demonstrated conditions. TGM builds on bilinear transduction [5]. The transductive reparameterization extends generalization beyond the in-distribution (ID) region by recasting part of the OOS region as a *delta-representable* range, where an unseen query $x^\star$ corresponds to novel combinations of anchors x^a and the relative change $\Delta x = x^\star - x^a$ that lie within their respective marginal distribution supports, illustrated in Fig. 1, although the pair was not observed together.

Diffusion and flow policies generate actions through repeated sampling of a denoising or velocity field. TGM evaluates this field at the anchor condition and adds a learned bilinear correction. This correction combines an embedding of the anchor condition and current sampling state with an embedding of the relative change to the query condition. Training uses pairs of demonstrated conditions. At deployment, TGM retrieves a nearby anchor and reuses a relative change learned from other training pairs. The corrected field then generates an action or action-chunk for the current conditions. Contact-rich manipulation creates an additional discontinuity problem. Small perturbations in object pose may result in abrupt changes in contact location or pushing direction, potentially violating the local smoothness of the state-to-action mapping assumed in direct policy prediction. TGM handles these tasks by generating the next state and recovering the corresponding action with a learned local inverse-dynamics model. The transductive operation then applies on state transitions rather than directly on a discontinuous action mapping. We implement TGM with Denoising Diffusion Probabilistic Models (DDPM) [1] and a velocity-field Flow Matching [2].

Why the parameterization can extrapolate. For a locally regular denoising field, a Taylor expansion around the anchor separates each change into a coefficient evaluated at the anchor and a polynomial feature of the relative change. The bilinear correction learns this product structure without derivative supervision. This interpretation requires bounded local derivatives, sufficient bilinear rank, and training coverage of the marginal supports of anchors and relative changes. It motivates TGM for delta-representable conditions. It does not establish a guarantee for arbitrary OOS queries, unsupported changes, or abrupt changes in the modeled transition.

We evaluate TGM on robotic tasks with unseen goals, obstacle configurations, states, and physical parameters. The experiments ask whether TGM improves on deterministic bilinear transduction, preserves multimodal behavior, and can handle contact-rich action discontinuities. On the four-task Bilinear Transduction benchmark, TGM matches the lowest reported error among learned methods on Reach and attains the lowest final-distance error on Push, Slider, and Adroit. On multimodal Dubins obstacle avoidance with OOS goals and the unseen obstacle centered at $y = -0.5$, it reduces OOS goal error from 1.01 ± 1.14 for conditional DDPM to 0.33 ± 0.44 and reduces the observed collision rate from 25% to 0%. In the PushT evaluation, TGM with next state prediction generation reduces OOS final distance from 55.28 ± 65.62 to 6.48 ± 6.56. In summary, the paper makes three contributions:

- It formulates OOS robotic imitation for unseen conditions that combine training-supported anchors and relative changes.
- It extends bilinear transduction to diffusion and flow policies by correcting an anchor-conditioned denoising or velocity field, with a Taylor interpretation that states the required regularity and coverage.

- It evaluates the learned policies in different robotic tasks across unseen goals, states, obstacles, and physical parameters, and further adapts transductive generation to next-state prediction for contact-rich manipulation.

2 Related Work

Transductive Learning and Extrapolation. Generalizing beyond the training support requires structural assumptions that constrain predictions under unseen inputs. Existing approaches exploit known symmetries, like geometric equivariance [3] and periodicity [4], to generalize predictions beyond the training support. However, the relevant symmetries may be unknown, and distribution shifts in conditions may extend beyond those captured by known symmetries. Netanyahu et al. [5] offer an alternative approach by introducing Bilinear Transduction, which predicts OOS inputs through anchor-displacement pairs. Simchowitz et al. [7] also establish guarantees for bilinear combinatorial extrapolation through matrix completion. Subsequent work by Song et al. [8] and Kim et al. [9] extend transduction to offline reinforcement learning by seeking ID anchors and transferring task-relevant latent transformations across contexts, respectively.

Conditional Generative Modeling in Robotics. Conditional generative models capture multimodal distributions over robot actions and trajectories. Janner et al. [10] introduce Diffuser, formulating planning as trajectory denoising with reward guidance and conditioning. Later, Chi et al. [11] propose Diffusion Policy, generating action chunks conditioned on observations. Ajay et al. [12] propose Decision Diffuser, generating state trajectories conditioned on returns, constraints, or skills and recovering actions through a learned inverse-dynamics model. To improve geometric generalization, Ze et al. [13] introduce 3D Diffusion Policy using compact point-cloud representations, and Ryu et al. [14] combine SE(3) bi-equivariance with diffusion-based pose generation in Diffusion-EDFs. Flow Matching [2] provides another approach to conditional action generation. Chisari et al. [15] investigate conditional flow matching with point-cloud in PointFlowMatch, while Zhang et al. [16] develop FlowPolicy, using consistency flow matching to enable single-step action generation. Beyond robotics, Rubbi et al. [6] improve generation under unseen conditions through mixture-conditioned flow matching. These studies motivate the use of expressive generative models while highlighting complementary roles for geometric representations and the treatment of conditions in generalization.

Imitation Learning under Distribution Shift. Distribution shift poses a central challenge in imitation learning, as policies may drift from the expert state distribution during execution or struggle to generalize to unseen environments and goals. Prior work mainly addresses this challenge through additional environment interactions, demonstration augmentation, and model-based control. Lee et al. [17] introduce Diff-DAgger, using diffusion-policy uncertainty to request expert corrections during rollouts and update the policy. Mandlekar et al. [18] propose MimicGen to augment demonstrations through trajectory adaptation and rollouts, while Xue et al. [19] use synthetic point-cloud editing in DemoGen to eliminate these rollouts. Wu et al. [20] augment diffusion policies with learned dynamic models for model-based planning, and Sun and Song [21] introduce Latent Policy Barrier to steer actions toward the expert distribution. However, these approaches primarily address state-distribution shift through corrective training, planning, or recovery, with several requiring additional interaction data. TGM takes a complementary approach, applying a delta-representable transductive parameterization to OOS conditioning inputs, including states, environment configurations, physical parameters, and goals, without additional data collection or simulation rollouts.

3 Preliminaries

Setup and Notation. The superscript target denotes target quantities, θ denotes learned parameters, and $\star$ denotes query samples and their associated quantities. Let $\mathcal{X} \subseteq \mathbb{R}^d$ and $\mathcal{Y} \subseteq \mathbb{R}^m$ denote the conditioning input space and output space, respectively. Each input $x \in \mathcal{X}$ induces a conditional target distribution $p^{\mathrm{target}}(y \mid x)$ over $y \in \mathcal{Y}$. Given a training dataset $D_{\mathrm{train}} = \{(x_i, y_i)\}_{i=1}^N$, coming from a training distribution $\mathcal{D}_{\mathrm{train}}$, our goal is to learn a conditional generative model $p_\theta(y \mid x)$ that approximates $p^{\mathrm{target}}(y \mid x)$.

Out-of-Support (OOS). We say that test distribution $\mathcal{D}_{\mathrm{test}}$ has an out-of-support distribution shift if there exists a subset $\mathcal{X}' \subseteq \mathcal{X}$ such that $\mathbb{P}_{x \sim \mathcal{D}_{\mathrm{test}}}[x \in \mathcal{X}'] > 0$ but $\mathbb{P}_{x \sim \mathcal{D}_{\mathrm{train}}}[x \in \mathcal{X}'] = 0$. Minimiz-

ing the risk under OOS shifted test distribution $\mathcal{D}_{\text{test}}$ is referred to as the *OOS extrapolation*. However, without additional assumptions, OOS extrapolation is ill-posed. **Delta-representable.** To make the *extrapolation* problem tractable, we adopt the transductive parameterization from prior work [5] to express delta-representable OOS queries through anchor-delta pairs. Under this reparameterization, the training distribution can be rewritten as a joint distribution over the anchor point $x^a \in \mathcal{X}$ and the displacement $\Delta x \in \Delta\mathcal{X}$ where $\Delta\mathcal{X} = \{x - x^a \mid x, x^a \in \mathcal{X}\}$. We assume that the test marginal distribution supports satisfy $\text{supp}(\mathcal{D}_{\text{test},X}) \subseteq \text{supp}(\mathcal{D}_{\text{train},X})$ and $\text{supp}(\mathcal{D}_{\text{test},\Delta X}) \subseteq \text{supp}(\mathcal{D}_{\text{train},\Delta X})$, while the joint distributions can be out-of-support: $\text{supp}(\mathcal{D}_{\text{test}}) \not\subseteq \text{supp}(\mathcal{D}_{\text{train}})$. An OOS query $x^\star$ is *delta-representable* if it can be written as $x^\star = x^a + \Delta x$ with x^a and Δx in these respective training marginal supports.

Bilinear Transduction. This delta-representable parameterization motivates the bilinear representation [5] to learn a function $\bar{h}_\theta(x^a, \Delta x) = \langle f_\theta(x^a), g_\theta(\Delta x)\rangle$ that approximates the ground-truth mapping $h^{\text{target}}(x^a + \Delta x)$ for an OOS input. Here, f_θ and g_θ are learnable embeddings that map the anchor and delta displacement from input space to a shared latent space $\mathbb{R}^r$ where r is the rank of the bilinear embeddings. For multidimensional output space in $\mathbb{R}^m$, it has the form

$$\left[\bar{h}_\theta(x^a, \Delta x)\right]_k = \langle f_{\theta,k}(x^a), g_{\theta,k}(\Delta x)\rangle \,. \tag{1}$$

Here, $[\bar{h}_\theta(x^a, \Delta x)]_k$ denotes the kth output component, from $k = 1, \ldots, m$, and $\langle\cdot,\cdot\rangle$ denotes the inner product. At training time, it samples the anchor (x^a, y^a) and query pairs $(x^\star, y^\star)$ from the training distribution and minimizes the loss between the predicted output and the ground truth output:

$$\mathcal{L}(\theta) = \mathop{\mathbb{E}}_{(x^a,y^a),(x^\star,y^\star)\sim\mathcal{D}_{\text{train}}} \left[\left\|\bar{h}_\theta(x^a, x^\star - x^a) - y^\star\right\|_2^2\right]. \tag{2}$$

At inference time, it selects an anchor x^a from the training distribution and predicts the output for the unseen input $x^\star$ by computing $\Delta x = x^\star - x^a$ and evaluating $\bar{h}_\theta(x^a, \Delta x)$. However, this output-wise bilinear representation neither captures dependencies across dimensions of the output latent space nor models multimodal distributions.

Conditional Generative Modeling. In contrast to these limitations of bilinear transduction, conditional generative models are capable of modeling high-dimensional multimodal distributions. Here, we focus on conditional denoising diffusion probabilistic models (DDPMs) [1] with y_0-prediction. We extend the formulation to flow matching with velocity prediction in Appendix A.1. Following the notation in Sec. 3, the model learns a conditional distribution $p_\theta(y \mid x)$ that approximates the target distribution $p^{\text{target}}(y \mid x)$. The model contains forward and reverse noise processes. Predictions are generated through iteratively sampling noise samples into clean samples that match the target conditional distribution. **Forward Process.** We sample a noisy output y_t from a Gaussian distribution conditioned on the clean output y_0. Given $y_0 \sim p^{\text{target}}(y \mid x)$, we have

$$y_t = \sqrt{\bar{\alpha}_t} y_0 + \sqrt{1 - \bar{\alpha}_t}\varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I_m), \quad q(y_t \mid y_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t} y_0, (1 - \bar{\alpha}_t) I_m). \tag{3}$$

where t is noise steps from 0 to T, β_t is the per-step noise variance, $\alpha_t = 1 - \beta_t$, and $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$, with $\bar{\alpha}_0 = 1$. The schedule is chosen so that $\bar{\alpha}_T \approx 0$, making y_T approximately standard Gaussian. Marginalizing over y_0 defines the conditional density $p_t(y_t \mid x)$. **Reverse Process.** Given a condition x, sampling starts from $y_T \sim \mathcal{N}(0, I_m)$ and proceeds through $t = T, \ldots, 1$, keeping x fixed. At each timestep, the denoising network predicts the clean output as $\hat{y}_0 = D_\theta(y_t, t, x)$. This estimate determines the mean of the reverse transition:

$$p_\theta(y_{t-1} \mid y_t, x) = \mathcal{N}\big(\mu_\theta(y_t, t, x), \sigma_t^2 I_m\big) \,. \tag{4}$$

The mean μ_θ is computed from y_t and $\hat{y}_0$ using the DDPM noise scheduler, which also determines the covariance $\sigma_t^2 I_m$. The estimated $\hat{y}_0$ output is recomputed at each sampling timestep. The network D_θ is trained by squared-error regression to y_0 using noisy samples from Eq. (3). Although the learned conditional generative model $p_\theta(y \mid x)$ performs well for in-distribution condition, it usually fails to generalize to unseen conditions $x^\star$ outside the training support. In the following section, we explain how we realize TGM that leverages the transductive reparameterization to improve the generalization of conditional generative models to unseen conditions.

4 Method

4.1 Transductive Generative Modeling

In this section, we first introduce TGM algorithm. Then we discuss the structural assumptions underlying TGM and how TGM can be applied to different robotics tasks.

Multi-output Bilinear Transduction. To apply bilinear transduction to DDPMs, we first propose a multi-output bilinear transduction that shares the latent feature across the multivariate output dimensions. Unlike the output-wise formulation in Eq. (1), which learns latent features per output dimension, our formulation combines shared latent features to produce the full output. This can be written as

$$\bar{h}_\theta^M(x^a, \Delta x) = W\left(f_\theta(x^a) \odot g_\theta(\Delta x)\right), \tag{5}$$

where $\odot$ denotes elementwise multiplication and W is a learnable weight matrix. Compared with Eq. (1), this replaces the per-output-dimension bilinear embeddings with r latent product features shared across all output dimensions, which is more training-efficient and can capture the interactions between output dimensions.

Transductive Parameterization. Using the notation in Sec. 3, $D_\theta(y_t, t, x)$ denotes the denoising network that predicts the clean output y_0 from the noisy output y_t, timestep t, and condition x. Its ideal regression target is $D_t^{\text{target}}(y_t \mid x) = \mathbb{E}[y_0 \mid y_t, x]$. For the Gaussian noising process, the ideal clean-output predictor satisfies

$$D_t^{\text{target}}(y_t \mid x) = \mathbb{E}[y_0 \mid y_t, x] = \frac{y_t + (1-\bar{\alpha}_t)\,\nabla_y \log p_t(y \mid x)\big|_{y=y_t}}{\sqrt{\bar{\alpha}_t}}. \tag{6}$$

This equation holds at DDPM timesteps with $0 < \bar{\alpha}_t < 1$. The learned denoiser D_θ approximates the target. Given a query input $x^\star$ and an anchor input x^a, the corresponding displacement is $\Delta x = x^\star - x^a$. For the conditional generative model introduced in Sec. 3, the log-conditional density $\log p_t(y_t \mid x^\star)$ at a DDPM timestep t can be expanded around the anchor input x^a to order q. Then we apply ∇_y, and substitute the result back into Eq. (6) to obtain:

$$D_t^{\text{target}}(y_t \mid x^\star) = D_t^{\text{target}}(y_t \mid x^a) + \sum_{i=1}^{q} \frac{1}{i!}\,\nabla_x^i D_t^{\text{target}}(y_t \mid x)\big|_{x=x^a}\left[(\Delta x)^{\otimes i}\right] + \mathcal{O}\Big(\|\Delta x\|_2^{q+1}\Big). \tag{7}$$

where $\nabla_x^i D_t^{\text{target}}$ is the ith-order derivative of the ideal denoiser with respect to the conditioning input x, evaluated at the anchor x^a. And $(\Delta x)^{\otimes i} = \Delta x \otimes \cdots \otimes \Delta x$ for i times, which is the i-fold tensor product of Δx.

Each term couples an anchor-dependent derivative feature with a displacement-dependent feature, providing the higher-order Taylor structure that can be approximated by the multi-output bilinear transduction in Eq. (5), we approximate the ideal predictor with a learned transductive predictor:

$$D_t^{\text{target}}(y_t \mid x^\star) \approx D_\theta(y_t, t, x^a) + \bar{h}_t^{M,\theta}(y_t, t, x^a, \Delta x) = D_\theta(y_t, t, x^a) + W\left(f_t^\theta(y_t, t, x^a) \odot g_t^\theta(\Delta x)\right). \tag{8}$$

Training Phase. Algorithm 1 summarizes the training procedure. Using Eq. (8), we jointly train the denoiser D_θ and the transductive correction to predict $\hat{y}_0$. At each iteration, we uniformly sample an anchor-query pair from the training data and a timestep t uniformly from 1 to T. We add independent Gaussian noise to the anchor and query outputs using the DDPM noise schedule at timestep t. The anchor loss $\mathcal{L}_a$ trains $D_\theta(y_t^a, t, x^a)$ to recover the clean anchor output y_0^a. For the query loss $\mathcal{L}_q$, we apply the denoiser to the noisy query output $y_t^\star$ while conditioning on the anchor x^a, then add the transductive correction. We compare this corrected prediction with the clean query output $y_0^\star$. Both losses use mean squared error, and we minimize $\mathcal{L}_q + \lambda_a \mathcal{L}_a$, where λ_a controls the weight of the anchor loss.

Inference Phase. Algorithm 2 summarizes the inference procedure. Given an unseen condition $x^\star$, we select an anchor x^a from the training anchor pool and compute Δx. The anchor and displacement remain fixed throughout generation. Starting from $y_T \sim \mathcal{N}(0, I_m)$, we perform the reverse process for $t = T, \ldots, 1$. At each timestep, we evaluate $D_\theta(y_t, t, x^a)$ and add the transductive correction to obtain the clean-output estimate $\hat{y}_0^\star$. This estimate determines the mean μ_t of the Gaussian denoising

Algorithm 1 TGM-DDPM Training

Require: Data $\mathcal{D}$; DDPM per-step noise variances $\{\beta_t\}_{t=1}^T$; anchor weight λ_a

$\alpha_t \leftarrow 1-\beta_t$, $\bar{\alpha}_t \leftarrow \prod_{s=1}^t \alpha_s$
for each optimization step **do**
  Sample anchor (x^a, y_0^a) and query $(x^\star, y_0^\star)$ from $\mathcal{D}$
  $\Delta x \leftarrow x^\star - x^a$
  Sample timestep $t \sim \mathrm{Unif}\{1,\dots,T\}$
  Sample $\varepsilon^a, \varepsilon^\star \overset{\text{i.i.d.}}{\sim} \mathcal{N}(0, I)$
  Apply forward noise: $y_t^\star \leftarrow \sqrt{\bar{\alpha}_t} y_0^\star + \sqrt{1-\bar{\alpha}_t}\varepsilon^\star$, $y_t^a \leftarrow \sqrt{\bar{\alpha}_t} y_0^a + \sqrt{1-\bar{\alpha}_t}\varepsilon^a$
  Compute the anchor loss: $\mathcal{L}_a \leftarrow \|D_\theta(y_t^a, t, x^a) - y_0^a\|_2^2$
  Compute the query loss with the transductive correction:
  $D_{t,q}^\theta \leftarrow D_\theta(y_t^\star, t, x^a)$ ▷ anchor-conditioned prediction
  $\bar{h}_{t,q}^{M,\theta} \leftarrow W\big[f_t^\theta(y_t^\star, t, x^a) \odot g_t^\theta(\Delta x)\big]$ ▷ transductive correction
  $\mathcal{L}_q \leftarrow \left\| D_{t,q}^\theta + \bar{h}_{t,q}^{M,\theta} - y_0^\star \right\|_2^2$
  Update θ using $\mathcal{L}_q + \lambda_a \mathcal{L}_a$
end for

Algorithm 2 TGM-DDPM Inference

Require: Query $x^\star$; anchor pool $\mathcal{X}_\mathcal{D}$; trained denoiser D_θ and transductive correction; DDPM per-step noise variances $\{\beta_t\}_{t=1}^T$

$\alpha_t \leftarrow 1-\beta_t$, $\bar{\alpha}_t \leftarrow \prod_{s=1}^t \alpha_s$, $\bar{\alpha}_0 \leftarrow 1$
$x^a \leftarrow \arg\min_{x \in \mathcal{X}_\mathcal{D}} d(x^\star, x)$; $\Delta x \leftarrow x^\star - x^a$
Sample $y_T \sim \mathcal{N}(0, I)$
for $t = T, \dots, 1$ **do**
  $D_{t,q}^\theta \leftarrow D_\theta(y_t, t, x^a)$
  $\bar{h}_{t,q}^{M,\theta} \leftarrow W\big[f_t^\theta(y_t, t, x^a) \odot g_t^\theta(\Delta x)\big]$
  $\hat{y}_0^\star \leftarrow D_{t,q}^\theta + \bar{h}_{t,q}^{M,\theta}$
  $\mu_t \leftarrow \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1-\bar{\alpha}_t}\hat{y}_0^\star + \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t} y_t$
  $\sigma_t \leftarrow \sqrt{\frac{\beta_t(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}}$
  if $t > 1$ **then**
    Sample noise $z \sim \mathcal{N}(0, I)$
    $y_{t-1} \leftarrow \mu_t + \sigma_t z$
  else
    $y_0 \leftarrow \mu_t$
  end if
end for
return y_0

sampling, while the DDPM noise schedule determines its variance. For $t > 1$, we sample y_{t-1} from this transition and use it as the noisy input at the next timestep. At $t = 1$, we return the transition mean as the final output y_0, without adding noise. The TGM-Flow formulation is provided in Appendix A.1.

4.2 TGM Taylor Interpretation and Structural Assumptions

We now provide a Taylor interpretation of the transductive parameterization and discuss how this informs the structural assumptions underlying TGM.

We interpret the transductive correction in Eq. (8) using the Taylor expansion in Eq. (7). For fixed y_t and t, each Taylor correction term couples an anchor-dependent derivative coefficient with $(\Delta x)^{\otimes i}$, the i-fold tensor product of the displacement Δx. This separation motivates the learned features $f_t^\theta(y_t, t, x^a)$ and $g_t^\theta(\Delta x)$, whose products are combined by W across output dimensions. Pfrommer et al. [22] also show that incorporating higher-order Taylor information into policy learning can improve robustness in imitation learning. In TGM, the anchor-displacement structure of the Taylor expansion is implicitly captured by a bilinear correction learned from anchor-query pairs, without explicit derivative supervision. This Taylor interpretation also highlights the following assumptions

Local regularity. For fixed y_t and t, we assume that the ideal denoiser $D_t^{\text{target}}(y_t \mid x)$ has continuous, bounded derivatives with respect to x up to order $q+1$ near the line segment connecting x^a and $x^\star$. These conditions justify the qth-order Taylor expansion in Eq. (7). **Sufficient rank.** We assume that r shared latent product features are sufficient to accurately approximate the denoising correction in Eq. (8). **Marginal support.** We adopt the delta-representability condition in Sec. 3: the selected anchor and its displacement to the query lie within their respective training marginal supports, although their combination may be unseen.

These conditions justify the Taylor expansion underlying TGM but may not hold strictly in practical tasks. Even when they are not fully satisfied, the learned bilinear correction may still provide a useful approximation, whose practical effectiveness we assess through TGM's empirical performance. We also discuss how TGM can be adapted for contact-rich tasks with discontinuous state-to-action mappings in Sec. 4.3.

4.3 TGM for Imitation Learning with Unseen Conditions

In this section, we describe how TGM can be applied to imitation learning under unseen conditions in robotic settings. We consider a dataset of expert demonstrations $\mathcal{D} = \{(\tau_i, g_i)\}_{i=1}^B$, where $\tau_i = \{(s_\ell, a_\ell)\}_{\ell=1}^{L_i}$ is a trajectory and g_i is its target goal. From each trajectory, we extract conditioning-

output pairs (x, y). For single step policy learning, $(x, y) = ((s_\ell, g_i), a_\ell)$. More generally, $x = (s_{\ell-H+1:\ell}, g_i)$ may contain a window of H states, and $y = a_{\ell:\ell+K-1}$ may contain a horizon of K actions. During training, we uniformly sample two state-goal conditioning inputs $x^a, x^\star \in \mathcal{X}$ from the different demonstration trajectories at same time steps and compute their displacement $\Delta x = x^\star - x^a$. At test time, we retrieve an anchor trajectory based on state-goal similarity and transduce the selected anchor point to generate an action for the unseen conditions.

Building on the analysis in Sec. 4.1, this direct policy-learning formulation would require the additional assumption that the conditional mapping is sufficiently smooth and continuous, allowing the Taylor-structured parameterization to estimate output changes transduced by local displacements in the conditioning space. However, this assumption may be violated in contact-rich robotic manipulation tasks, where contact can introduce discontinuities in the state-to-action mapping. To address this challenge, inspired by Decision Diffuser [12], we perform Transductive Conditional Generative Modeling in the state space to predict the next state condition on current state, which often holds the smoothness assumption. We then infer the corresponding actions using a learned local inverse-dynamics model, which takes the current state as the reference origin and maps the predicted relative state transition to the action required to realize it. Thereby avoiding direct transduction over a potentially discontinuous policy mapping. This formulation provides a more general framework that can be applied to TGM in various robotic tasks, including contact-rich tasks with discontinuous state-to-action mappings.

5 Experiments

Our empirical evaluation aims to answer the following questions: (1) Does TGM generalize better than the bilinear transduction method [5]? (2) How effectively does TGM model and generalize to datasets with multimodal distributions? (3) How does TGM help improve generalization when the underlying policy exhibits discontinuous action mappings?

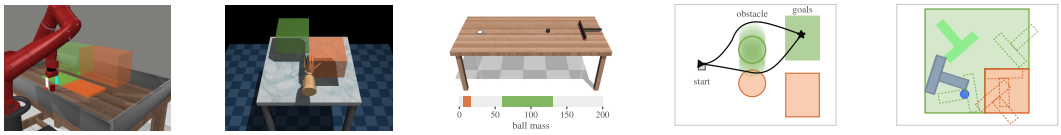
5.1 Baselines

To answer these questions, we compare TGM with several baseline methods, the Bilinear Transduction method [5], Shortest-Path Flow Matching (SP-FM) [6], and Conditional DDPM in various simulation tasks with unseen conditions. Bilinear Transduction is a baseline for learning predictors that generalize to unseen conditions and provides open-source implementations and datasets for four robotic manipulation tasks. SP-FM learns a condition-dependent Gaussian-mixture base distribution and a conditional flow field. We adapt SP-FM to condition on robot observations and predict actions. Conditional DDPM follows the Diffusion Policy formulation [11] when it predicts actions directly from observations. It follows the Decision Diffuser formulation [12] when it predicts future states and uses the learned inverse-dynamics model to recover actions. For a fair comparison, all methods use the same observation and action representations, matched network backbones unless otherwise specified: MLPs for single-step action prediction and U-Nets for action-chunk prediction. All generative models use 10 sampling steps as denoising steps for DDPM and explicit Euler integration steps for flow-based methods.

5.2 Experimental Results

Bilinear Transduction Benchmark. The open-source Bilinear Transduction benchmark released by Netanyahu et al. [5] comprises four robotic manipulation tasks: Reach, Push, Slider, and Adroit. Reach and Push evaluate extrapolation to OOS goals adapted from the Meta-World Benchmark [23]. For Reach, training goals are sampled from the green region, $[0, 0.4] \times [0.7, 0.9] \times [0.05, 0.3]$, and OOS goals from the orange region, $[-0.4, 0] \times [0.7, 0.9] \times [0.05, 0.3]$. For Push, the corresponding training regions are $[0, 0.3] \times [0.5, 0.7] \times \{0.01\}$ and test regions are $[-0.3, 0] \times [0.5, 0.7] \times \{0.01\}$. The Adroit task extends OOS goals evaluation to high-dimensional state in $\mathbb{R}^{39}$ and action spaces $\mathbb{R}^{30}$. Slider instead assesses extrapolation to OOS dynamics by conditioning the model on unseen ball masses. Specifically, the model is trained on masses in $[60, 130]$ and evaluated on the disjoint range $[5, 15]$, which induces different contact dynamics.

We evaluate TGM on these tasks using the same dataset and simulation protocols established in the Bilinear Transduction benchmark. Table 1 reports the mean and standard deviation over three



(a) Reach / Push (b) Adroit (c) Slider (d) Dubins (e) PushT

Figure 2: (a)–(e) Training conditions in green, out-of-support conditions in orange.

Table 1: OOS final distance to the goal across Bilinear Transduction Benchmark manipulation tasks. Entries report means with standard deviations in subscripts.

Task	Expert†	Bilinear Transduction†	SP-FM	Conditional DDPM	TGM-Flow	TGM-DDPM
Reach	$0.006_{0.008}$	$\mathbf{0.007}_{0.006}$	$0.010_{0.010}$	$0.131_{0.104}$	$\mathbf{0.007}_{0.007}$	$\mathbf{0.007}_{0.007}$
Push	$0.012_{0.001}$	$0.022_{0.016}$	$0.186_{0.131}$	$0.081_{0.104}$	$0.015_{0.005}$	$\mathbf{0.014}_{0.003}$
Slider	$0.105_{0.066}$	$0.163_{0.126}$	$0.488_{0.377}$	$0.738_{0.315}$	$0.070_{0.044}$	$\mathbf{0.047}_{0.055}$
Adroit	$0.035_{0.015}$	$0.198_{0.255}$	$0.261_{0.407}$	$0.246_{0.169}$	$0.062_{0.029}$	$\mathbf{0.054}_{0.030}$

training seeds and evaluation samples, where each seed is evaluated on the 50 OOS goal samples. The Expert† and Bilinear Transduction† columns are taken from Table 5 in the appendix of Netanyahu et al. [5]. TGM-DDPM also achieves lower mean final distance on Push, Slider, and Adroit than the best Bilinear Transduction results reported in their Table 1. These results show the benefit of transductive generative modeling on the more challenging OOS tasks in this benchmark.

Table 2: Dubins: OOS Performance Across Obstacle Positions. Entries report means with standard deviations in subscripts.

	$o_y = -0.1$		$o_y = -0.2$		$o_y = -0.3$		$o_y = -0.4$		$o_y = -0.5$	
Method	Dist. ↓	Coll. ↓	Dist. ↓	Coll. ↓	Dist. ↓	Coll. ↓	Dist. ↓	Coll. ↓	Dist. ↓	Coll. ↓
Bilinear Transduction	$0.907_{0.973}$	5.8%	$0.917_{1.135}$	5.0%	$0.757_{1.152}$	0.8%	$0.561_{0.825}$	1.7%	$0.568_{0.727}$	3.3%
SP-FM	$1.326_{1.232}$	25.8%	$1.381_{1.166}$	33.3%	$1.362_{1.103}$	34.2%	$1.558_{1.104}$	34.2%	$1.810_{1.140}$	30.8%
Conditional DDPM	$0.962_{1.015}$	31.7%	$1.047_{1.003}$	33.3%	$0.969_{1.043}$	24.2%	$0.995_{1.038}$	27.5%	$1.013_{1.140}$	25.0%
TGM-Flow	$0.456_{0.529}$	2.5%	$\mathbf{0.462}_{0.624}$	**1.7%**	$\mathbf{0.328}_{0.549}$	1.7%	$0.430_{0.707}$	1.7%	$0.739_{0.934}$	2.5%
TGM-DDPM	$\mathbf{0.387}_{0.320}$	**1.7%**	$0.468_{0.693}$	2.5%	$0.441_{0.685}$	**0.0%**	$\mathbf{0.324}_{0.429}$	**0.0%**	$\mathbf{0.328}_{0.439}$	**0.0%**

Task with Multimodal Policy (Dubins Car). To answer the second question, we consider the Dubins car obstacle-avoidance task, in which the model predicts the linear velocity $v \in \mathbb{R}^1$ and angular velocity $\omega \in \mathbb{R}^1$ conditioned on Dubins car states $x = [p_x, p_y, \theta] \in \mathbb{R}^3$, goal position $x_g = [p_{gx}, p_{gy}] \in \mathbb{R}^2$ and obstacles $o_y \in \mathbb{R}^1$. Multimodality arises when passing on either side of an obstacle results in collision-free paths of similar lengths given the same state and goal. We evaluate the model under a joint OOS shift in both goal and obstacle positions, measuring final distance to the goal and collision rate with the obstacle. We split the 3,390 demonstrations into 3,053 training trajectories and 337 validation trajectories. Training goals lie within $[1.0, 2.0] \times [0.2, 1.5]$, with obstacle y-coordinates in $\{0.0, 0.1, \ldots, 0.9\}$, whereas OOS test goals lie within $[1.0, 2.0] \times [-1.5, -0.2]$, with the obstacle y-coordinate varied over $\{-0.1, -0.2, \ldots, -0.5\}$. For Bilinear Transduction and both TGM variants, we clamp Δx to better satisfy the marginal-support assumption in Sec. 4.2. The clamp threshold is based on the range of training displacements. Table 2 reports the mean and standard deviation of final distance across four training seeds, each evaluated on 30 OOS episodes per obstacle position. We further evaluate the OOS performance of TGM at $o_y = -0.5$ on two subsets of OOS trajectories: (1) Delta-representable trajectories, which contain only displacements within the training marginal support, and (2) Out-of-Delta-representable Support (OODS) trajectories, which involve out-of-support displacements, either due to goals outside the training displacement support or as a result of rollout into states requiring unsupported displacements. The number of delta-representable trajectories varies across methods, as the displacements encountered depend on the rollouts generated by each method. The numbers of delta-representable trajectories are 55, 19, and 42 for bilinear transduction, TGM-Flow, and TGM-DDPM, respectively, out of 120 OOS test. Both TGM variants achieve lower final distance than bilinear transduction on delta-representable trajectories (0.091 ± 0.006 for TGM-Flow and 0.093 ± 0.006 for TGM-DDPM versus 0.258 ± 0.319 for bilinear transduction), while TGM-DDPM also achieves lower final distance on OODS trajectories

(0.454 ± 0.501 versus 0.830 ± 0.863). These results suggest that TGM more effectively models multimodal distributions and generalizes to unseen conditions in the delta-representable range.

Table 3: PushT: ID vs. OOS goals. Entries report means with standard deviations in subscripts.

	ID		OOS	
Method	Dist. (px) ↓	θ err. (rad) ↓	Dist. (px) ↓	θ err. (rad) ↓
Bilinear Transduction	$25.914_{28.283}$	$0.414_{0.710}$	$20.916_{25.053}$	$0.356_{0.717}$
SP-FM	$25.447_{32.568}$	$0.255_{0.373}$	$55.913_{75.892}$	$0.307_{0.395}$
Conditional DDPM	$20.410_{34.895}$	$0.162_{0.439}$	$55.280_{65.617}$	$0.508_{0.761}$
TGM-Flow	$11.233_{10.052}$	$0.088_{0.151}$	$9.746_{7.579}$	$0.058_{0.072}$
TGM-DDPM	$\mathbf{8.885}_{10.238}$	$\mathbf{0.036}_{0.047}$	$\mathbf{6.479}_{6.558}$	$\mathbf{0.038}_{0.047}$

Task with Discontinuity and Contact (PushT). As discussed in Sec. 4.3, TGM can be applied to state prediction in discontinuous and contact-rich settings. We evaluate this capability on the PushT task, where the policy determines the pusher contact point $p_c \in \mathbb{R}^2$ and pushing direction $p_d \in \mathbb{R}^2$ conditioned on the current pose $x = [p_x, p_y, \theta] \in \mathbb{R}^3$ and goal poses $x_g = [p_{gx}, p_{gy}, \theta_g] \in \mathbb{R}^3$ of the T-shaped object. The direct state-to-action mapping can be discontinuous because of contact dynamics and abrupt changes in the required pusher position. To obtain a smoother prediction target, TGM and all baselines instead predict an 8-step state-transition chunk conditioned on the current state and goal. The predicted transitions are converted into actions using a shared inverse-dynamics model implemented as a conditional DDPM and the first 4 actions are executed before the policy replans. This design holds action recovery and execution fixed across all methods in Table 3.

We partition demonstrations according to the target T object's position (p_{gx}, p_{gy}) in pixel coordinates. Goals for which both coordinates exceed 280 pixels are held out from training. We split the 180 demonstrations into 160 training trajectories and 20 validation trajectories. Each method is trained with three seeds and evaluated on 20 ID and 20 OOS episodes per seed. Table 3 reports the final pixel distance and angular error in radians. Final PushT rollout states are visualized in Appendix A.2 (Fig. 3). TGM achieved better results in both ID and OOS Goals. By adapting TGM to next-state prediction with learned inverse dynamics, this experiment demonstrates how TGM can improve generalization in contact-rich settings with discontinuous state-to-action mappings.

6 Conclusion

We introduced Transductive Generative Modeling (TGM), a framework that combines transductive reparameterization with conditional generative modeling for robot policy learning. TGM represents an unseen condition using a training-supported anchor and displacement, then applies a learned bilinear correction to the anchor-conditioned denoising or velocity field. We evaluate TGM across diverse robotic tasks involving different types of unseen conditions and demonstrate robust generalization beyond the training support. However, results also show that policy rollouts can leave the delta-representable range, where TGM's underlying support assumption no longer holds. This motivates future work on multi-anchor selection and OOS detection to improve robustness and guide policy rollouts toward supported regions. Another promising direction is to extend TGM to visuomotor policies by learning a delta-representable latent space to support generalization to unstructured high-dimensional conditions.

Acknowledgments

We thank Junyan Jiang for helpful discussions and insightful feedback. This work was supported in part by funding from the Office of Naval Research under grant N00014-23-1-2590, the Department of Energy under grant DE-SC0026902, the National Science Foundation under grant DMS-2310831, DMS-2428059, IIS-2435696, IIS-2440954, Two Sigma Investments LP, and LG Management Development Institute AI Research. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of the sponsors.

References

[1] J. Ho, A. N. Jain, and P. Abbeel. Denoising diffusion probabilistic models. In *NeurIPS*, 2020.

[2] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le. Flow matching for generative modeling. In *ICLR*, 2023.

[3] A. Simeonov, Y. Du, A. Tagliasacchi, J. B. Tenenbaum, A. Rodriguez, P. Agrawal, and V. Sitzmann. Neural descriptor fields: SE(3)-equivariant object representations for manipulation. In *ICRA*, 2022. doi:10.1109/icra46639.2022.9812146.

[4] W. Cho, M. Jo, K. Lee, and N. Park. Neural functions for learning periodic signal. In *ICLR*, 2025. doi:10.48550/arXiv.2506.09526.

[5] A. Netanyahu, A. Gupta, M. Simchowitz, K. Zhang, and P. Agrawal. Learning to extrapolate: A transductive approach. In *ICLR*, 2023. doi:10.48550/arXiv.2304.14329.

[6] A. Rubbi, A. Akbarnejad, M. V. Sanian, A. Y. Parast, H. Asadollahzadeh, A. Amani, N. Akhtar, S. Cooper, A. Bassett, P. Liò, L. Paavolainen, S. Vakili, and M. Lotfollahi. Shortest-path flow matching with mixture-conditioned bases for OOD generalization to unseen conditions. arXiv:2601.11827, 2026.

[7] M. Simchowitz, A. Gupta, and K. Zhang. Tackling combinatorial distribution shift: A matrix completion perspective. In *COLT*, 2023. doi:10.48550/arXiv.2307.06457.

[8] Y. Song, D. Lee, and G. Kim. Compositional conservatism: A transductive approach in offline reinforcement learning. In *ICLR*, 2024. doi:10.48550/arXiv.2404.04682.

[9] J. Kim, D. Kim, M. Hong, and S. Oh. Compositional transduction with latent analogies for offline goal-conditioned reinforcement learning. In *ICML*, 2026. doi:10.48550/arXiv.2605.20609.

[10] M. Janner, Y. Du, J. Tenenbaum, and S. Levine. Planning with diffusion for flexible behavior synthesis. In *ICML*, 2022. doi:10.48550/arXiv.2205.09991.

[11] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. C. Burchfiel, and S. Song. Diffusion Policy: Visuomotor Policy Learning via Action Diffusion. In *Proceedings of Robotics: Science and Systems*, Daegu, Republic of Korea, July 2023. doi:10.15607/RSS.2023.XIX.026.

[12] A. Ajay, Y. Du, A. Gupta, J. Tenenbaum, T. Jaakkola, and P. Agrawal. Is conditional generative modeling all you need for decision-making? In *ICLR*, 2023. doi:10.48550/arXiv.2211.15657.

[13] Y. Ze, G. Zhang, K. Zhang, C. Hu, M. Wang, and H. Xu. 3D diffusion policy: Generalizable visuomotor policy learning via simple 3D representations. In *RSS*, 2024. doi:10.15607/rss.2024.xx.067.

[14] H. Ryu, J. Kim, H. An, J. Chang, J. Seo, T. Kim, Y. Kim, C. Hwang, J. Choi, and R. Horowitz. Diffusion-EDFs: Bi-equivariant denoising generative modeling on SE(3) for visual robotic manipulation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 18007–18018, June 2024.

[15] E. Chisari, N. Heppert, M. Argus, T. Welschehold, T. Brox, and A. Valada. Learning robotic manipulation policies from point clouds with conditional flow matching. In P. Agrawal, O. Kroemer, and W. Burgard, editors, *Proceedings of The 8th Conference on Robot Learning*, volume 270 of *Proceedings of Machine Learning Research*, pages 982–993. PMLR, 2025. URL `https://proceedings.mlr.press/v270/chisari25a.html`.

[16] Q. Zhang, Z. Liu, H. Fan, G. Liu, B. Zeng, and S. Liu. FlowPolicy: Enabling fast and robust 3D flow-based policy via consistency flow matching for robot manipulation. In *AAAI*, 2025. doi:10.1609/aaai.v39i14.33617.

[17] S.-W. Lee, X. Kang, and Y.-L. Kuo. Diff-DAgger: Uncertainty estimation with diffusion policy for robotic manipulation. In *ICRA*, 2025. doi:10.1109/ICRA55743.2025.11127730.

[18] A. Mandlekar, S. Nasiriany, B. Wen, I. Akinola, Y. Narang, L. Fan, Y. Zhu, and D. Fox. MimicGen: A data generation system for scalable robot learning using human demonstrations. In J. Tan, M. Toussaint, and K. Darvish, editors, *Proceedings of The 7th Conference on Robot Learning*, volume 229 of *Proceedings of Machine Learning Research*, pages 1820–1864. PMLR, 06–09 Nov 2023. URL https://proceedings.mlr.press/v229/mandlekar23a.html.

[19] Z. Xue, S. Deng, Z. Chen, Y. Wang, Z. Yuan, and H. Xu. DemoGen: Synthetic demonstration generation for data-efficient visuomotor policy learning. In *RSS*, 2025. doi:10.15607/rss.2025.xxi.157.

[20] R. Wu, H. Chen, M. Zhang, H. Lu, Y. Li, and Y. Li. Neural dynamics augmented diffusion policy. In *ICRA*, 2025. doi:10.1109/ICRA55743.2025.11128651.

[21] Z. Sun and S. Song. Latent policy barrier: Learning robust visuomotor policies by staying in-distribution. In *NeurIPS*, 2025. doi:10.52202/085713-5797.

[22] D. Pfrommer, T. T. C. K. Zhang, S. Tu, and N. Matni. TaSIL: Taylor series imitation learning. In *NeurIPS*, 2022. doi:10.52202/068431-1466.

[23] T. Yu, D. Quillen, Z. He, R. Julian, K. Hausman, C. Finn, and S. Levine. Meta-World: A benchmark and evaluation for multi-task and meta reinforcement learning. In L. P. Kaelbling, D. Kragic, and K. Sugiura, editors, *Proceedings of the Conference on Robot Learning*, volume 100 of *Proceedings of Machine Learning Research*, pages 1094–1100. PMLR, 2020. URL https://proceedings.mlr.press/v100/yu20a.html.

A Appendix

A.1 TGM-Flow: Formulation and Algorithms

Algorithm 3 TGM-Flow Training

Require: Data $\mathcal{D}$; anchor weight λ_a
for each optimization step **do**
  Sample anchor (x^a, y^a) and query $(x^\star, y^\star)$ from $\mathcal{D}$
  $\Delta x \leftarrow x^\star - x^a$
  Sample time $t \sim \mathrm{Unif}[0, 1)$
  Sample $\varepsilon^a, \varepsilon^\star \overset{\text{i.i.d.}}{\sim} \mathcal{N}(0, I)$
  Interpolate between noise and data:
  $y_t^a \leftarrow (1-t)\varepsilon^a + t y^a$
  $y_t^\star \leftarrow (1-t)\varepsilon^\star + t y^\star$
  Velocity targets: $v^a \leftarrow y^a - \varepsilon^a$, $v^\star \leftarrow y^\star - \varepsilon^\star$
  $\mathcal{L}_a \leftarrow \|V_\theta(y_t^a, t, x^a) - v^a\|_2^2$ ▷ Anchor loss
  $V_{t,q}^\theta \leftarrow V_\theta(y_t^\star, t, x^a)$
  $\bar{h}_{t,q}^{M,\theta} \leftarrow W\left[f_t^\theta(y_t^\star, t, x^a) \odot g_t^\theta(\Delta x)\right]$
  $\mathcal{L}_q \leftarrow \left\| V_{t,q}^\theta + \bar{h}_{t,q}^{M,\theta} - v^\star \right\|_2^2$ ▷ Query loss
  Update θ using $\mathcal{L}_q + \lambda_a \mathcal{L}_a$
end for

Algorithm 4 TGM-Flow Inference

Require: Query $x^\star$; anchor pool $\mathcal{X}_\mathcal{D}$; trained velocity predictor V_θ and transductive correction; number of Euler steps $K \geq 1$
$x^a \leftarrow \arg\min_{x \in \mathcal{X}_\mathcal{D}} d(x^\star, x)$; $\Delta x \leftarrow x^\star - x^a$
Sample $y^{(0)} \sim \mathcal{N}(0, I)$
$\Delta t \leftarrow 1/K$
for $k = 0, \ldots, K-1$ **do**
  $t_k \leftarrow k\Delta t$
  $V_{k,q}^\theta \leftarrow V_\theta(y^{(k)}, t_k, x^a)$
  $\bar{h}_{k,q}^{M,\theta} \leftarrow W\left[f_{t_k}^\theta(y^{(k)}, t_k, x^a) \odot g_{t_k}^\theta(\Delta x)\right]$
  $u_k^\star \leftarrow V_{k,q}^\theta + \bar{h}_{k,q}^{M,\theta}$
  $y^{(k+1)} \leftarrow y^{(k)} + \Delta t\, u_k^\star$
end for
return $y^{(K)}$

TGM-Flow replaces the denoiser in Eq. (8) with a velocity predictor while retaining the anchor-conditioned predictor and bilinear correction. For flow matching, we write the Gaussian conditional probability path as $y_t = a_t y + b_t \varepsilon$, where y is the target output, $\varepsilon \sim \mathcal{N}(0, I_m)$, and $t \in [0, 1]$ is continuous flow time. The coefficients a_t and b_t are continuously differentiable noise schedules. For DDPM, $a_t = \sqrt{\bar{\alpha}_t}$ and $b_t = \sqrt{1 - \bar{\alpha}_t}$ with t decreasing toward the target sample, which recovers Eq. (6) from Eq. (10). For rectified flow, $a_t = t$ and $b_t = 1 - t$ with t increasing toward it. For times with $a_t, b_t > 0$, the conditional marginal velocity satisfies

$$u_t^{\text{target}}(y_t \mid x) = c_t y_t + d_t \nabla_{y_t} \log p_t(y_t \mid x), \tag{9}$$

where $c_t = \dot{a}_t / a_t$ and $d_t = c_t b_t^2 - b_t \dot{b}_t$. The Gaussian denoising identity underlying Eq. (6) has the following form for this path:

$$\mathbb{E}[y \mid y_t, x] = \frac{y_t + b_t^2 \nabla_{y_t} \log p_t(y_t \mid x)}{a_t}. \tag{10}$$

Since Eq. (9) relates the target velocity to the log-density gradient $\nabla_{y_t} \log p_t(y_t \mid x)$ in the same way that Eq. (6) relates the target predictor, the same Taylor interpretation in Sec. 4.1 applies to the target velocity. During inference, sampling starts from $y_{t=0} \sim \mathcal{N}(0, I_m)$ and proceeds from $t = 0$ to 1 by numerically integrating the corrected velocity field,

$$\frac{dy_t}{dt} = u_t^\theta(y_t \mid x), \qquad t : 0 \to 1. \tag{11}$$

Training and sampling. Algorithms 3 and 4 use the rectified-flow schedules $a_t = t$ and $b_t = 1 - t$. Here, y^a and $y^\star$ are target outputs, V_θ is the velocity predictor, and $y^{(k)}$ is the sample after k Euler steps. The anchor and conditioning displacement remain fixed during sampling.

A.2 PushT OOS Final States.

Figures 3a and 3b show the final state of every OOS episode of the first training seed for TGM-DDPM and TGM-Flow, complementing the statistics in Table 3. Each panel is the 512×512 px board: the initial position is randomized for each episode. The green square is the range of training goals, the orange square is the unseen OOS goal corner ($x, y > 280$ px), the green outline is the target goal pose, the grey T is the final pose of the block, and the blue dot is the final pusher position.



(a) TGM-DDPM (b) TGM-Flow

Figure 3: PushT: final states of the 20 OOS episodes (seed 0), TGM-DDPM (left) and TGM-Flow (right).